

Atualização de Registros via API

Avançando no ciclo de manutenção dos dados: aprenda a modificar registros existentes com segurança, usando requisições HTTP e o REST Client para testar o comportamento da sua API.

REST API · PATCH

Missão: Atualizando Registros com Segurança no Backend

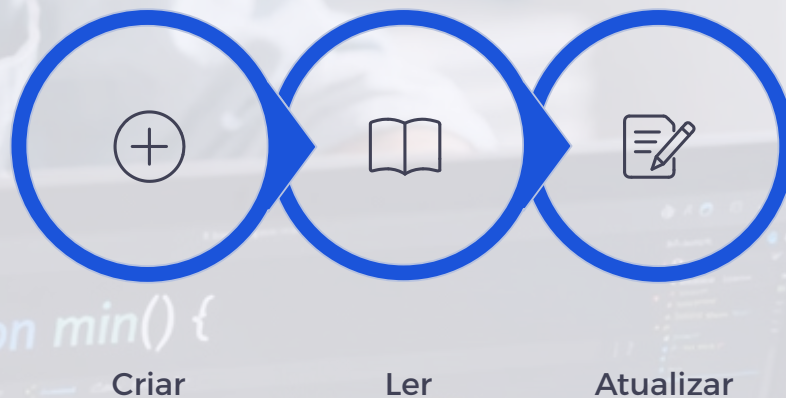
Onde estamos no CRUD

Até aqui, já criamos registros e realizamos a leitura dos dados cadastrados. Agora, o objetivo é permitir que um registro existente seja **alterado** por meio de uma requisição HTTP.

Por que isso importa?

Os dados de uma aplicação raramente permanecem imutáveis. Um produto pode ter seu nome ajustado, uma descrição pode ser corrigida, um status pode mudar e uma informação cadastrada incorretamente pode precisar de revisão.

Por isso, a API deve oferecer um fluxo **claro, seguro e previsível** para modificar informações já persistidas no banco de dados.



O ciclo CRUD avança etapa a etapa. Neste material, chegamos à terceira fase: a atualização controlada de registros já existentes.

Justificativa Pedagógica e Objetivos de Aprendizagem

A implementação da atualização aprofunda a compreensão do **ciclo de vida de uma entidade** dentro da aplicação. O estudante passa a lidar com identificação por id, payload de alteração, validação de existência do registro e separação de responsabilidades entre as camadas da aplicação.

1

Explicar

A diferença entre criação e atualização de registros

2

Implementar

Um endpoint de atualização em uma API REST completa

3

Validar

Se o item existe antes de alterá-lo, retornando erro adequado

4

Testar

O fluxo completo usando um arquivo `.http` no REST Client

O que Significa Atualizar um Registro

Criação vs. Atualização

Atualizar um registro **não é criar outro item** no banco de dados.

- **Na criação:** enviamos dados novos para que o sistema gere um novo registro com um novo `id`.
- **Na atualização:** informamos qual registro já existente deve ser alterado e quais campos receberão novos valores.

O `id` do registro permanece o mesmo após a atualização. Apenas os dados internos são modificados.

Exemplo Conceitual

Produto cadastrado:

ID: 10

Nome: Arroz Integral

Preço: 8.50

Atualização desejada:

Preço: 9.20

Produto após atualização:

ID: 10

Nome: Arroz Integral

Preço: 9.20

PUT ou PATCH? Escolhendo o Método Correto

PUT – Substituição Completa

Representa uma substituição mais completa do recurso. A ideia é enviar uma **nova versão inteira** do registro.

```
PUT /products/10
{
```

PUT – Substituição Completa - **continuação**

```
"name": "Arroz Integral Tipo 1",  
"description": "Pacote 1kg",  
"price": 9.20  
}
```

PATCH – Atualização Parcial

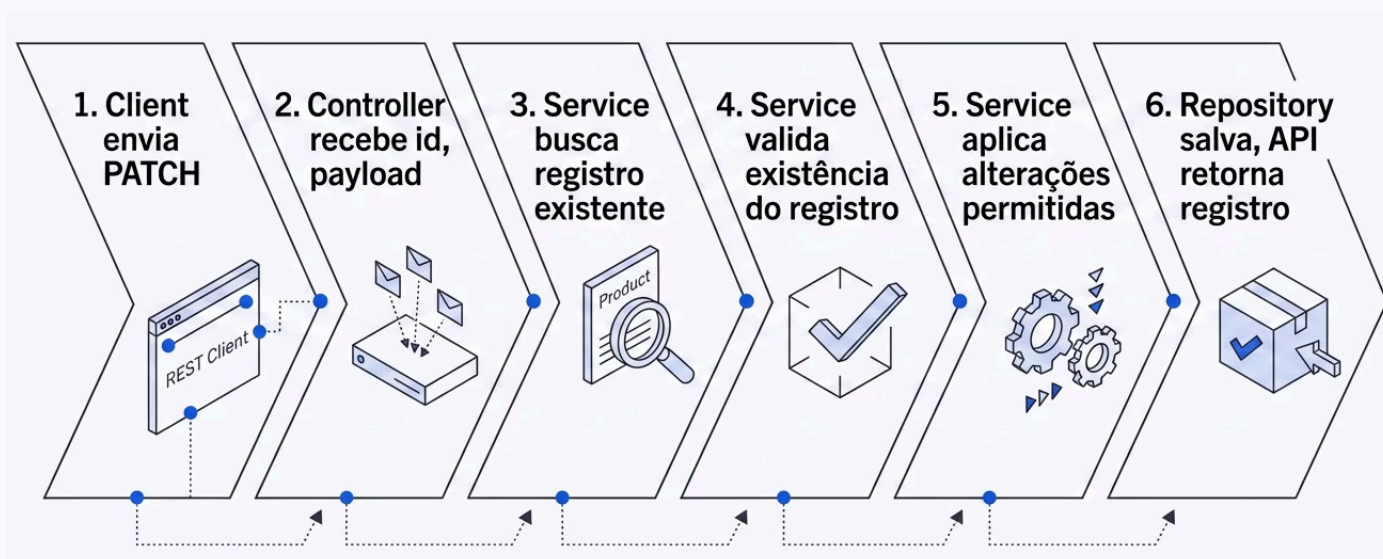
Representa uma atualização parcial. A ideia é enviar **apenas os campos** que serão alterados.

```
PATCH /products/10  
{  
  "price": 9.20  
}
```

Para este material, a sugestão é usar **PATCH**, pois ele combina melhor com atualizações parciais e facilita a compreensão inicial dos alunos.

Fluxo Completo da Atualização

O fluxo esperado reforça a responsabilidade de cada camada da aplicação. Cada componente tem um papel bem definido no processo de atualização.



Controller

Recebe a requisição HTTP, extrai o **id** da rota e o **body** da requisição. Não contém lógica de negócio.

Service

Concentra a regra da atualização: busca o registro, valida existência, aplica as mudanças permitidas.

Repository

Conversa com o banco de dados, persistindo a nova versão do registro de forma segura.

Implementação Passo a Passo



Passo 1 — DTO de Atualização

Crie um DTO próprio para atualização com campos opcionais:

```
export class UpdateProductDto {  
  name?: string;  
  description?: string;  
  price?: number;  
}
```



Passo 2 — Método no Controller

O controller recebe o `id` pela rota e o corpo pelo `body`:

```
@Patch('/:id')  
update(  
  @Param('id') id: string,  
  @Body() updateProductDto: UpdateProductDto,  
) {  
  return this.productsService.update(id, updateProductDto);  
}
```



Passo 3 — Regra no Service

O service segue uma sequência clara: buscar, validar, aplicar e salvar:

```
async update(id: string, updateProductDto: UpdateProductDto) {  
  const product = await this.productsRepository.findOne({  
    where: { id } });  
}
```



Passo 3 — Regra no Service - **continuação**

```
if (!product) {  
    throw new NotFoundException('Produto não encontrado.');
```



```
    Object.assign(product, updateProductDto);  
  
    return this.productsRepository.save(product);  
}
```

Testando com o REST Client

O arquivo `.http` deve cobrir múltiplos cenários para garantir que a API se comporta corretamente em todas as situações.

UPDATE 1 — Caminho Feliz

```
### Atualiza um produto existente  
PATCH {{baseUrl}}/products/{{productId}}  
Content-Type: {{contentType}}  
  
{  
    "name": "Produto atualizado",  
    "description": "Registro alterado no teste."  
}
```

Verifique: a API retornou sucesso? Os campos aparecem com novos valores? O `id` permaneceu o mesmo?

UPDATE 2 — Confirmar pela Leitura

```
### Consulta o produto atualizado  
GET {{baseUrl}}/products/{{productId}}
```

Testar atualização não é apenas verificar se o `PATCH` respondeu. É necessário confirmar se a mudança foi **realmente persistida**.

UPDATE 3 — Registro Inexistente

Tenta atualizar produto inexistente

PATCH {{baseUrl}}/products/00000000-0000-0000-0000-000000000000

Content-Type: {{contentType}}

```
{
  "name": "Produto que não existe"
}
```

Comportamento esperado: 404 Not Found. A API não deve criar um novo produto.

UPDATE 4 — Payload Parcial

PATCH {{baseUrl}}/products/{{productId}}

Content-Type: {{contentType}}

```
{
  "description": "Apenas descrição alterada."
}
```

Verifique se apenas a descrição mudou, sem afetar os demais campos.

UPDATE 5 — Payload Vazio

PATCH {{baseUrl}}/products/{{productId}}

Content-Type: {{contentType}}

```
{}
```

Decida com a turma: aceitar e retornar sem alterações, ou rejeitar exigindo ao menos um campo?

Cuidados Importantes na Atualização

Nem todo campo de uma entidade deve ser livremente alterado pelo usuário. O DTO de atualização ajuda a controlar quais propriedades podem ser recebidas na requisição.



Campos Protegidos

Nunca devem ser alterados pelo usuário via API:

- `id`
- `createdAt`
- `updatedAt`
- `ownerId`



Campos com Regra

Exigem validação adicional antes de salvar:

- Status calculado
- Campos derivados
- Campos controlados por regra de negócio



Campos Permitidos

Definidos explicitamente no DTO de atualização:

- `name`
- `description`
- `price`

É perigoso permitir que qualquer propriedade enviada no JSON seja aplicada diretamente à entidade sem validação. Use sempre um DTO específico para atualização.

Checklist de Implementação e Checkpoint de Entendimento

✓ Checklist de Implementação

Antes de considerar o material concluído, verifique:

- ☒ Endpoint `PATCH /products/:id` foi criado
- ☐ Controller recebe corretamente o id da rota
- ☐ Controller recebe corretamente o body da requisição
- ☐ Existe um DTO próprio para atualização
- ☐ Service busca o registro antes de alterar
- ☐ Service retorna 404 quando o registro não existe
- ☐ A atualização não cria um novo registro
- ☐ A alteração é persistida no banco de dados
- ☐ REST Client possui testes para sucesso e erro
- ☐ A leitura por ID confirma a atualização realizada



Checkpoint de Entendimento

Ao final deste material, o estudante deve conseguir responder:

- Qual é a diferença entre **criar** e **atualizar** um registro?
- Por que a atualização precisa receber um `id`?
- Qual é a diferença entre `PUT` e `PATCH`?
- Por que o service deve verificar se o registro existe antes de salvar?
- Por que testar o `GET` depois do `PATCH` ajuda a confirmar o comportamento?
- Quais campos **não deveriam** ser atualizados livremente pelo usuário?

Resumo Final e Próximos Passos

Atualizar significa modificar um recurso existente, mantendo sua identidade e alterando apenas os campos permitidos.

Neste material, implementamos o fluxo completo de atualização de registros via API. Os principais aprendizados foram:

Identidade Preservada

O **id** do registro permanece o mesmo após a atualização. Apenas os dados internos são modificados.

Validação Obrigatória

O service deve sempre verificar se o registro existe antes de aplicar qualquer alteração.

Separação de Responsabilidades

Controller recebe, Service processa, Repository persiste. Cada camada com seu papel.

DTO Específico

Um DTO próprio para atualização controla quais campos podem ser modificados pelo usuário.



O próximo passo será implementar a **exclusão segura de registros**, fechando o ciclo básico do CRUD com atenção aos riscos de remover dados de forma indevida.